

Graph Neural Networks

CPSC483: Deep Learning on Graph-Structured Data

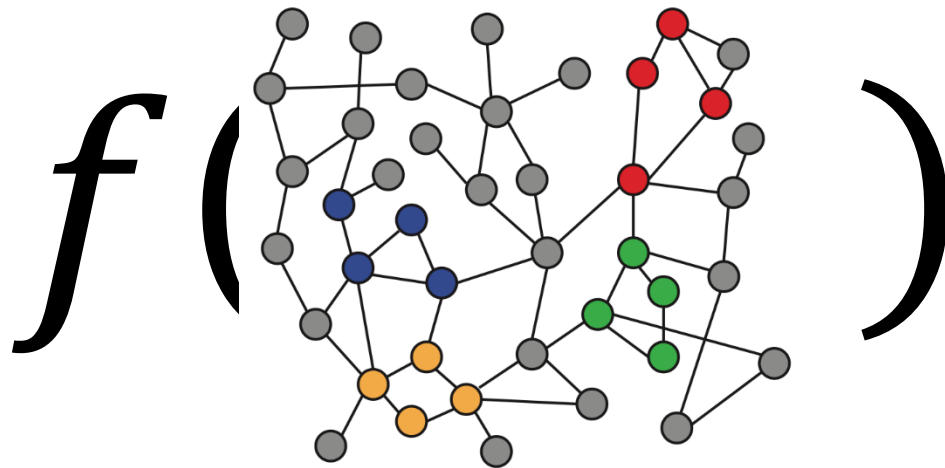
Rex Ying

Readings

- Readings are updated on the website (syllabus page)
- **Lecture 2 readings:** PageRank and Personalized PageRank (PPR)
- **Lecture 3 readings:**
 - [Graph representation learning: methods and application](#)
 - Inductive Representation Learning for Large Graphs ([GraphSAGE](#))

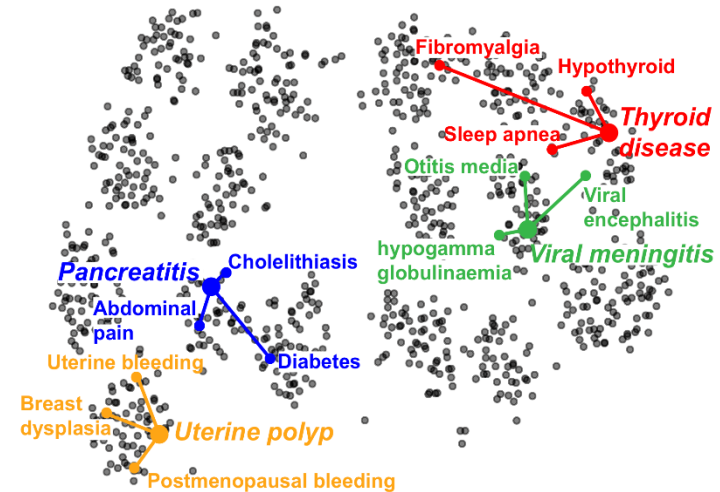
Node Embeddings

- Intuition: Map nodes to d-dimensional **embeddings** (which are “**representations**” of nodes) such that similar nodes in the graph are embedded close together



Input graph

=



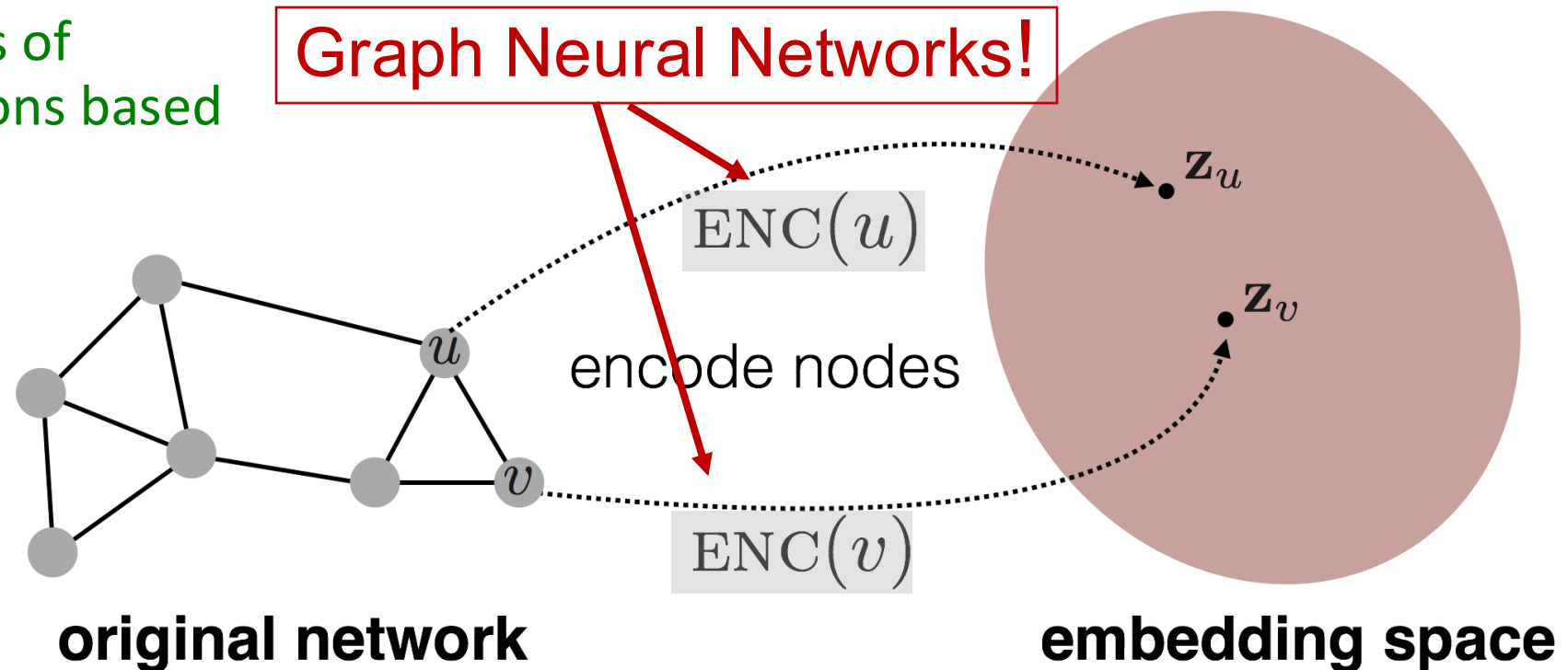
2D node embeddings

How to learn the mapping function f ?

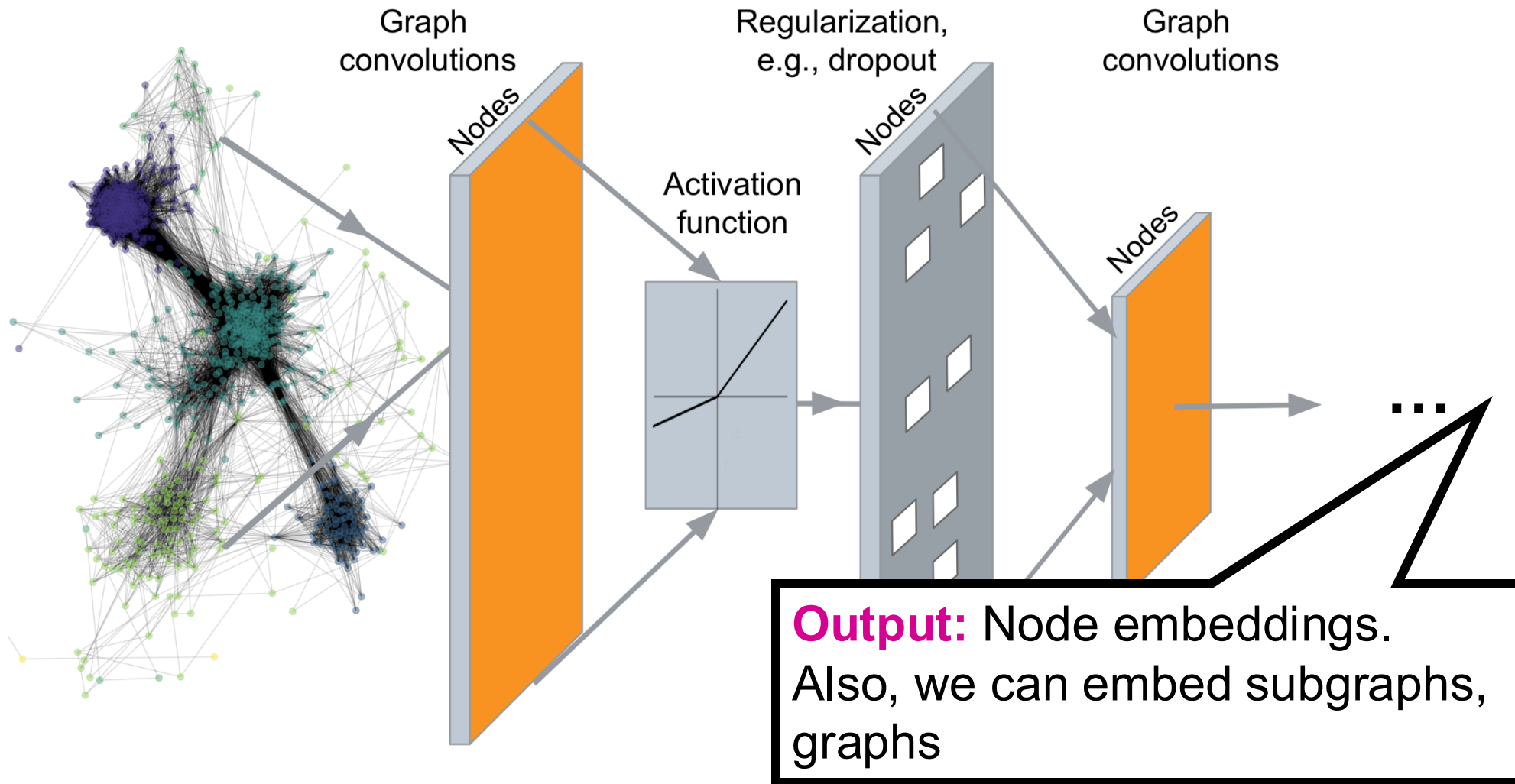
Deep Graph Encoders (1)

- **Today:** We will now discuss deep methods based on **graph neural networks (GNNs)**:

$\text{ENC}(\cdot)$ = multiple layers of non-linear transformations based on graph structures



Deep Graph Encoders (2)



Outline of Today's Lecture

- 1. Basics of deep learning**
- 2. Deep learning for graphs**

Outline of Today's Lecture

1. Basics of deep learning

2. Deep learning for graphs

Basics of Deep Learning

Machine Learning as Optimization (1)

- **Supervised learning:** we are given input x , and the goal is to predict label y
- **Input x can be:**
 - Vectors of real numbers
 - Sequences (natural language)
 - Matrices (images)
 - Graphs (potentially with node and edge features)
- **We formulate the task as an optimization problem**

Machine Learning as Optimization (2)

- **Formulate the task as an optimization problem:**

$$\min_{\Theta} \mathcal{L}(y, f(x))$$

← **Objective function**

- Θ : a set of **parameters** we optimize
 - Could contain one or more scalars, vectors, matrices ...
 - E.g. $\Theta = \{Z\}$ in the shallow encoder (the embedding lookup)

- $\mathcal{L}$: **loss function**. Example: L2 loss

$$\mathcal{L}(y, f(x)) = \|y - f(x)\|_2$$

- Other common loss functions:
 - L1 loss, huber loss, max margin (hinge loss), cross entropy ...
 - See <https://pytorch.org/docs/stable/nn.html#loss-functions>

Loss Function Example: Cross Entropy (1)

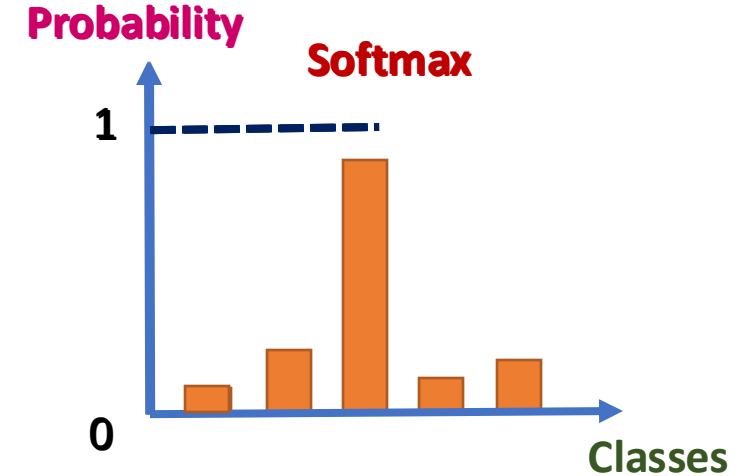
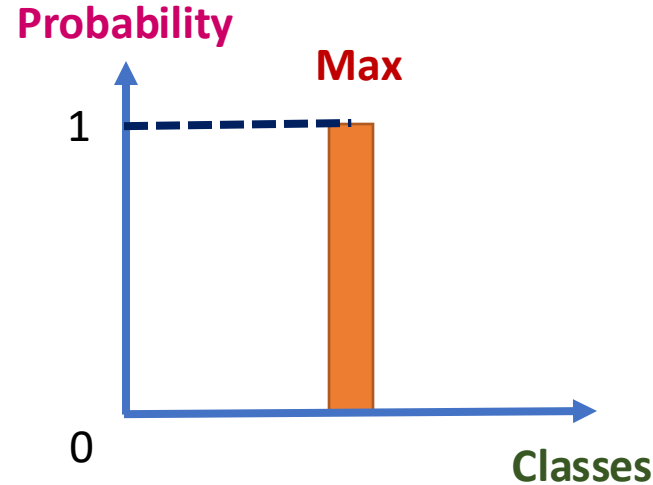
- One common loss for **classification**: cross entropy (CE). Supposed that:
- $f(\mathbf{x})$ is the output of a model
 - E.g. $f(\mathbf{x}) = [0.1, 0.1, 0.6, 0.2, 0]$
- Label $\mathbf{y}$ is a categorical vector (**one-hot** encoding)
 - E.g. $\mathbf{y} = [0, 0, 1, 0, 0]^T$ $\mathbf{y}$ belongs to class “3” because the 3rd entry is 1
- $\text{Softmax}(f(\mathbf{x}))_i = \frac{e^{f(\mathbf{x})_i}}{\sum_{j=1}^C e^{f(\mathbf{x})_j}}$ $\longrightarrow$ $f(\mathbf{x})_i$ denotes i -th coordinate of the vector $f(\mathbf{x})$
 - Where C is the number of classes. ($C = 5$ in this example)
 - E.g. $f(\mathbf{x}) = [0.1767, 0.1767, 0.2914, 0.1953, 0.1599]^T$

Softmax

- Softmax is a **differentiable** (or soft) version of the max function

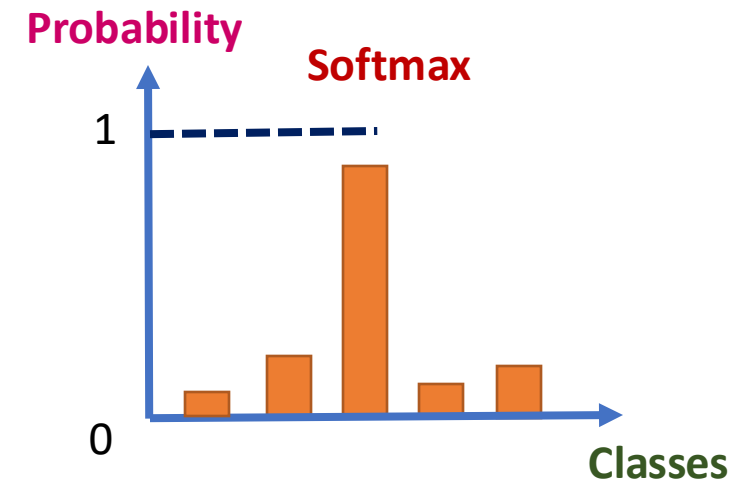
- $$\text{Softmax}(f(\mathbf{x}))_i = \frac{e^{f(\mathbf{x})_i}}{\sum_{j=1}^C e^{f(\mathbf{x})_j}}$$

- Where C is the number of classes. ($C = 5$ in this example)
- E.g. $f(\mathbf{x}) = [0.1767, 0.1767, 0.2914, 0.1953, 0.1599]^T$



Loss Function Example: Cross Entropy (2)

- $\text{CE}(\mathbf{y}, f(\mathbf{x})) = -\sum_{i=1}^C (\mathbf{y}_i \log f(\mathbf{x})_i)$
 - $\mathbf{y}_i, f(\mathbf{x})_i$ are the **actual** and **predicted** value of the i -th class.
 - **Intuition:** the lower the loss, the closer the prediction is to one-hot
- In classification, $\mathbf{y}$ is **one-hot**, whereas $f(\mathbf{x})$ is the output of a softmax
 - The summation in CE only has **1 non-zero term**
- Total loss over all training examples
 - $\mathcal{L} = \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{T}} \text{CE}(\mathbf{y}, f(\mathbf{x}))$
 - $\mathcal{T}$: training set containing all pairs of data and labels $(\mathbf{x}, \mathbf{y})$



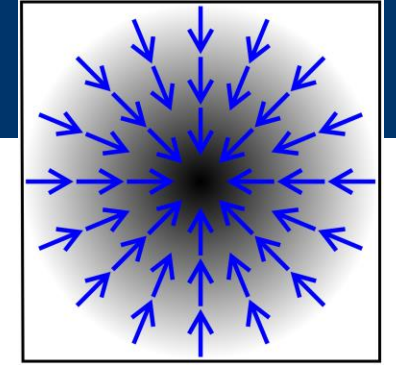
Machine Learning as Optimization (1)

- How to optimize the **objective function**?
- Non-gradient approaches
 - Bayesian optimization, Gaussian processes, Simulated annealing, Evolutionary algorithms

In deep learning, we use gradient approaches for scalability

- Therefore, we require the loss function $\mathcal{L}$ to be **differentiable**
 - There are ways to tackle optimization for non-differentiable functions:
 - Straight-through estimator (Gumbel Softmax)
 - Reinforce algorithm, or more generally, reinforcement learning

Machine Learning as Optimization (2)



<https://en.wikipedia.org/wiki/Gradient>

- How to optimize the **objective function**?

- **Gradient vector**: Direction and rate of fastest increase

$$\nabla_{\Theta} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \Theta_1}, \frac{\partial \mathcal{L}}{\partial \Theta_2}, \dots \right) \longleftarrow \text{Partial derivative}$$

- $\Theta_1, \Theta_2 \dots$: components of Θ
- Recall **directional derivative** of a multi-variable function (e.g. $\mathcal{L}$) along a given vector represents the instantaneous rate of change of the function along the vector.

Why Gradient?

What are pros and cons of optimizing in the direction of gradients?

Gradient Descent

- **Iterative algorithm:** repeatedly update weights in the (opposite) direction of gradients until convergence

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}$$

- **Training:** Optimize Θ iteratively
 - **Iteration:** 1 step of gradient descent
- **Learning rate (LR) η :**
 - Hyperparameter that controls the size of gradient step
 - Can vary over the course of training (LR scheduling)
- **Ideal termination condition: 0 gradient**
 - In practice, we stop training if it no longer improves performance on **validation set** (part of dataset we hold out from training)

Stochastic Gradient Descent (SGD)

- **Problem with gradient descent:**

- Exact gradient requires computing $\nabla_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$, where $\mathbf{x}$ is the **entire** dataset!
 - This means summing gradient contributions over all the points in the dataset
 - Modern datasets often contain billions of data points
 - Extremely expensive for every gradient descent step

- **Solution: Stochastic gradient descent (SGD)**

- At every step, pick a different **minibatch** $\mathcal{B}$ containing a subset of the dataset, and use it as input $\mathbf{x}$

Minibatch SGD

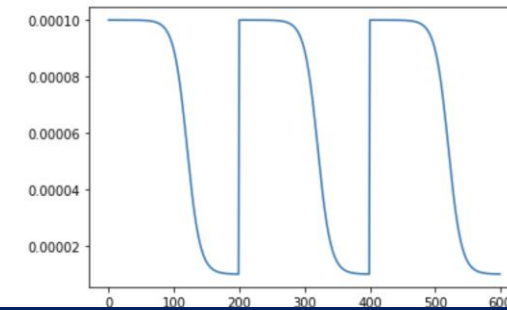
- **Concepts:**

- **Batch size:** the number of data points in a minibatch
 - E.g. number of nodes for node classification task
- **Iteration:** 1 step of SGD on a minibatch
- **Epoch:** one full pass over the dataset (# iterations is equal to ratio of dataset size and batch size)
- **SGD is unbiased estimator of full gradient:**
 - But there is no guarantee on the rate of convergence
 - In practice often requires tuning of learning rate

Variants of SGD

- Common optimizer that improves over SGD:

- Adam, Adagrad, Adadelata, RMSprop ...
- Adam is a good choice in many cases
- [Automatic optimization](#) in PyTorch Lightning could be good for many models
- Learning rate annealing schedules
- Hyper-parameter search over optimizer / scheduler parameters



Example:
tanh scheduler

Convergence;
generalization

Why such annealing is beneficial?

```
from timm.scheduler import TanhLRScheduler
def configure_optimizers(self):
    optimizer=...
    scheduler=TanhLRScheduler(optimizer,...)
    return[optimizer],[{"scheduler":scheduler, "interval": "epoch"}]

def lr_scheduler_step(self,scheduler,metric):
    scheduler.step(epoch=self.current_epoch)
```

Use in PyTorch Lightning

Neural Network Function (1)

- **Objective:** $\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$
- In deep learning, the function f can be very complex
- To start simple, consider linear function

$$f(\mathbf{x}) = \mathbf{W} \cdot \mathbf{x}, \quad \Theta = \{\mathbf{W}\}$$

- If f returns a scalar, then $\mathbf{W}$ is a learnable **vector**

$$\nabla_{\mathbf{W}} f = \left(\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}, \frac{\partial f}{\partial w_3} \dots \right)$$

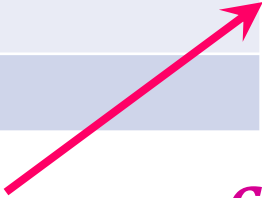
- If f returns a vector, then $\mathbf{W}$ is the **weight matrix**

$$\nabla_{\mathbf{W}} f = \mathbf{W}^T$$

Neural Network Function (2)

Generalized Derivative of f w.r.t. X	Scalar	Vector	Matrix
Scalar	Scalar	Vector	Matrix
Vector	Vector	Matrix	Tensors ☹️
Matrix	Matrix	Tensors ☹️	Tensors ☹️

Jacobian matrix of f



- Higher order (>2) tensor operations are not very intuitive
- GPU and CUDA kernels are mostly optimized for matrix computations

Back-propagation

- **How about a more complex function:**

$$f(\mathbf{x}) = a = W_2(\underbrace{W_1 \mathbf{x}}_{\mathbf{z}}), \quad \Theta = \{W_1, W_2\}$$

- Recall **chain rule**:

$$\text{E.g. } \nabla_{\mathbf{x}} f = \frac{\partial a}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{x}}$$

We define:

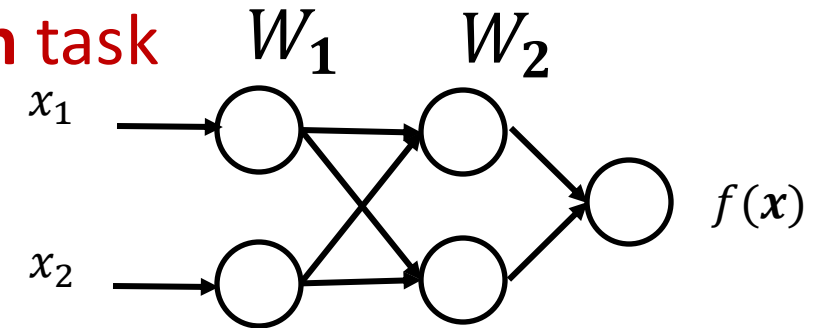
$$\mathbf{z} = W_1 \mathbf{x}$$

$$a = f(\mathbf{x}) = W_2 \mathbf{z}$$

- **Back-propagation**: Use of **chain rule** to propagate gradients of intermediate steps, and finally obtain gradient of $\mathcal{L}$ w.r.t. Θ

Back-propagation Example (1)

- **Example:** Simple 2-layer linear network, **regression** task
- $f(\mathbf{x}) = a = W_2 \mathbf{z} = \underbrace{W_2 (W_1 \mathbf{x})}_{\mathbf{z}}$
- $\mathcal{L} = \sum_{(x,y) \in \mathcal{B}} \left\| (y - f(\mathbf{x})) \right\|_2$ sums the L2 loss in a minibatch $\mathcal{B}$
- **Hidden layer:** intermediate representation for input $\mathbf{x}$
 - Here we use $\mathbf{z} = W_1 \mathbf{x}$ to denote the hidden layer



Back-propagation Example (2)

- **Forward propagation:**

Compute loss starting from input

$$\bullet \quad \mathbf{x} \xrightarrow{\text{Multiply } W_1} \mathbf{z} \xrightarrow{\text{Multiply } W_2} a \xrightarrow{\text{Loss}} \mathcal{L}$$

- **Back-propagation to compute gradient of**
 $\Theta = \{W_1, W_2\}$

- Start from loss, compute the gradient

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial W_2},$$

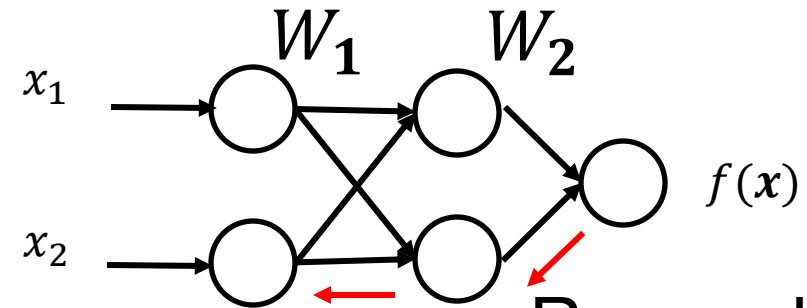
Compute backwards

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial W_1}$$

Compute

What should you do??

General method: Decompose by indices;
Case-specific: Composition of linear functions



Remember:

$$f(\mathbf{x}) = W_2(W_1\mathbf{x})$$

$$\mathbf{z} = W_1\mathbf{x}$$

$$a = W_2\mathbf{z}$$

Back-propagation: Concrete Example (1)

- Suppose that (minibatch of size 1)

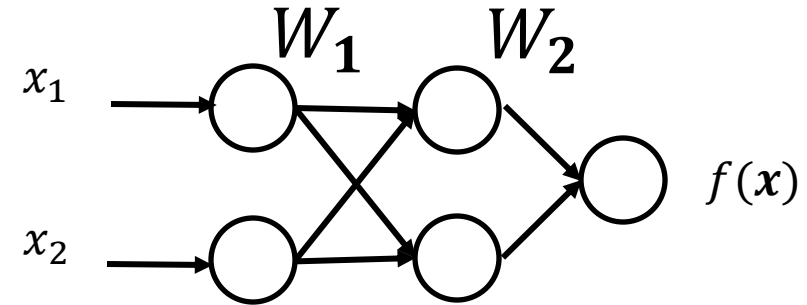
- **Features:** $\mathbf{x} = (x_1 \ x_2)^T = (0.8 \ 1.1)^T$

- **Label:** $y = 1.00$

- **Weights:** $W_1 = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix}, W_2 = \begin{pmatrix} 0.5 & 0.6 \end{pmatrix}$

- **Model output:** $a = f(\mathbf{x}) = W_2 W_1 \mathbf{x} = \begin{pmatrix} 0.5 & 0.6 \end{pmatrix} \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix} \begin{pmatrix} 0.8 \\ 1.1 \end{pmatrix} = 0.558$

- **Loss function:** $\mathcal{L} = \|y - a\| = (0.558 - 1)^2 = 0.1954$



Back-propagation: Concrete Example (2)

- To calculate gradients of W_2 :
- Recall that
 - $\mathcal{L} = \|y - a\|_2$
 - $a = W_2 \mathbf{z}$
 - $\mathbf{z} = W_1 \mathbf{x}$
- Apply chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial W_1} = -2(y - a) \cdot \frac{\partial a}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial W_1}$$

$$= -2(y - a) \cdot (xW_2)^T = \begin{pmatrix} -0.3536 & -0.4862 \\ -0.4243 & -0.5834 \end{pmatrix}$$

Non-linearity

- Note that in $f(\mathbf{x}) = W_2(W_1\mathbf{x})$, W_2W_1 is another matrix (or vector, if we do binary classification and output only 1 logit)
- Hence $f(\mathbf{x})$ is still linear w.r.t. $\mathbf{x}$ no matter how many weight matrices we compose

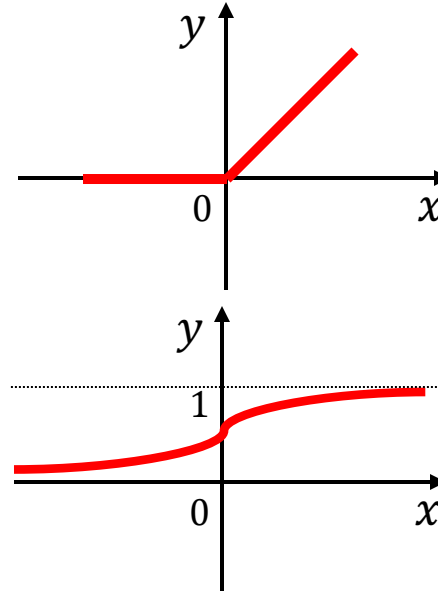
- **Introduce non-linearity:**

- **Rectified linear unit (ReLU)**

$$\text{ReLU}(x) = \max(x, 0)$$

- **Sigmoid**

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



ReLU

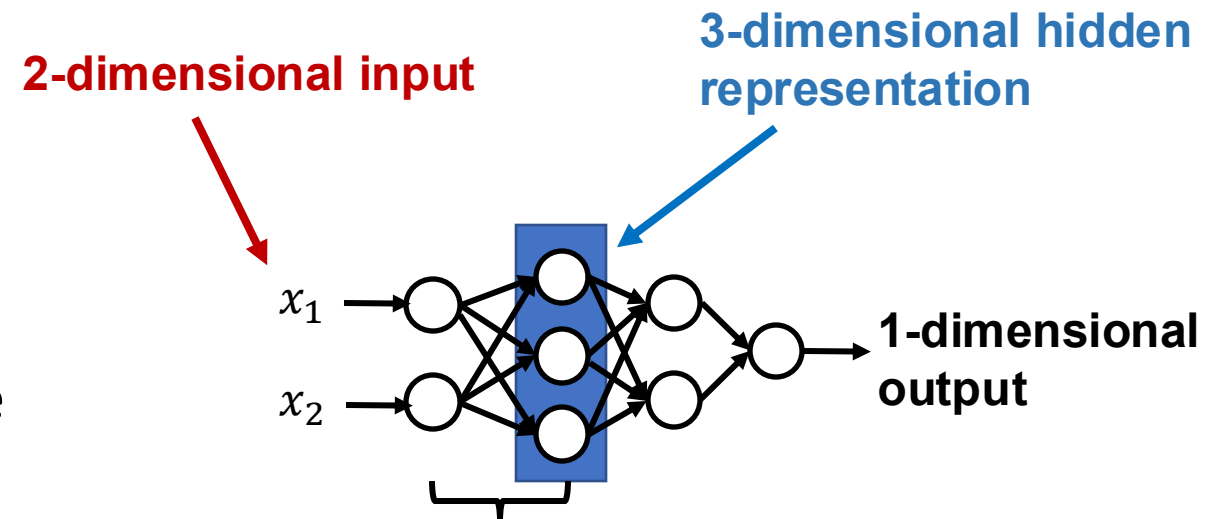
Sigmoid

Multi-layer Perceptron (MLP)

- Each layer of MLP combines linear transformation and non-linearity:

$$\mathbf{x}^{(l+1)} = \sigma(W_l \mathbf{x}^{(l)} + b^l)$$

- where W_l is weight matrix that transforms hidden representation at layer l to layer $l + 1$
- b^l is bias at layer l , and is added to the linear transformation of $\mathbf{x}$
- σ is non-linearity function (e.g., sigmoid)
- Suppose $\mathbf{x}$ is 2-dimensional, with entries x_1 and x_2



Every layer:
Linear transformation +
non-linearity

What is Universal Approximation?
What are the caveats?

Embedding dimension; Generalization;
Convergence; Invariances

Summary

- **Objective function:**

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$$

- f can be a simple linear layer, an MLP, or other neural networks (e.g., a GNN later)
- Sample a minibatch of input $\mathbf{x}$
- **Forward propagation:** compute $\mathcal{L}$ given $\mathbf{x}$
- **Back-propagation:** obtain gradient $\nabla_{\Theta} \mathcal{L}$ using a chain rule
- Use **stochastic gradient descent (SGD)** to optimize for Θ over many iterations

Outline of Today's Lecture

1. Basics of deep learning

2. Deep learning for graphs

Deep Learning for Graphs

Content

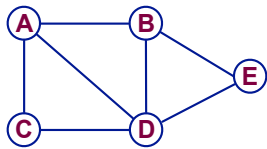
- **Local network neighborhoods:**
 - Describe aggregation strategies
 - Define computation graphs
- **Stacking multiple layers:**
 - Describe the model, parameters, training
 - How to fit the model?
 - Simple example for unsupervised and supervised training

Setup (recap)

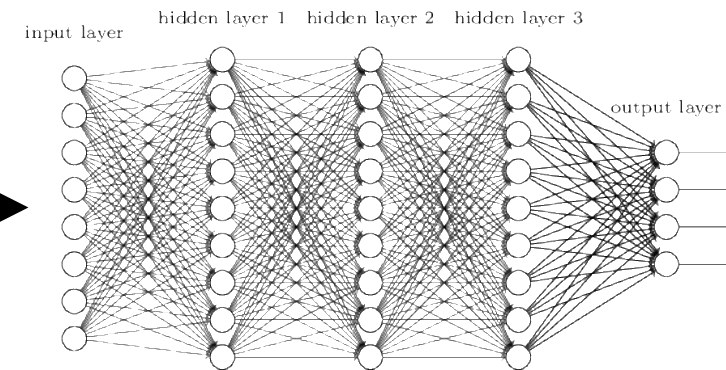
- **Assume we have a graph G :**
 - V is the **vertex set**
 - A is the **adjacency matrix** (assume binary)
 - $X \in \mathbb{R}^{d \times |V|}$ is a matrix of **node features**
 - v : a node in V ; $N(v)$: the set of neighbors of v .
 - **Node features:**
 - Social networks: User profile, User image
 - Biological networks: Gene expression profiles, gene functional information
 - When there is no node feature in the graph dataset:
 - Indicator vectors (one-hot encoding of a node)
 - Vector of constant 1: $[1, 1, \dots, 1]$

A Naïve Approach

- Join adjacency matrix and features
- Feed them into a deep neural net:



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0

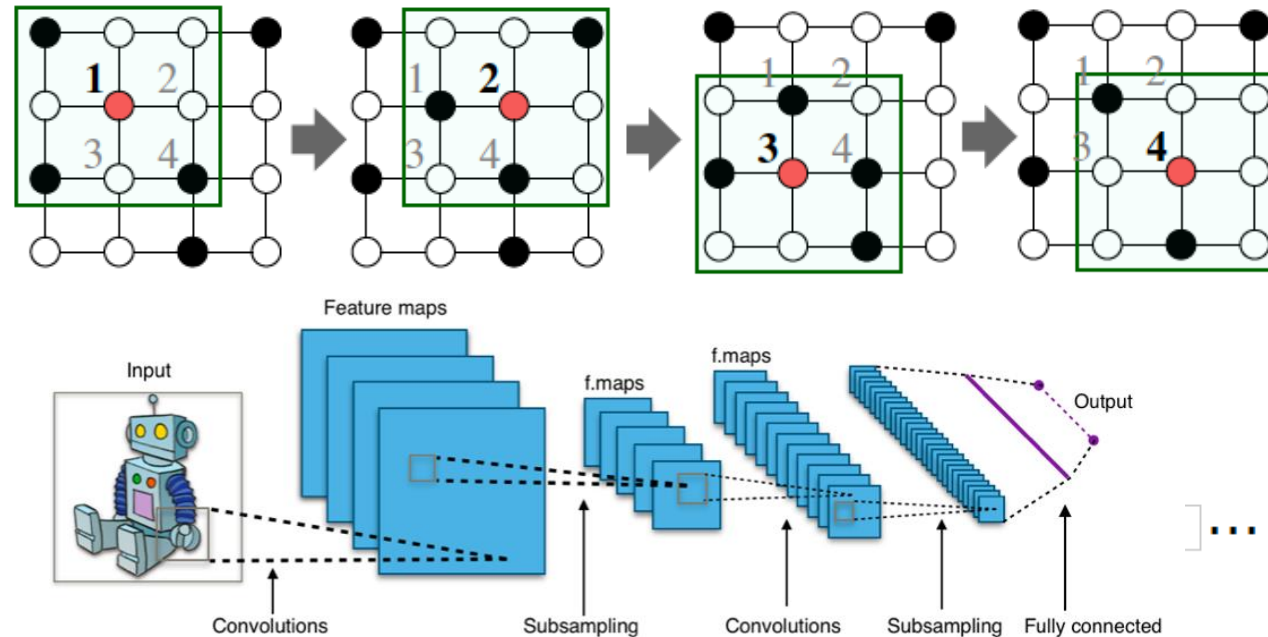


What are the issues?

- $O(|V|)$ parameters
- Not applicable to graphs of different sizes
- Sensitive to node ordering

Idea: Convolutional Networks

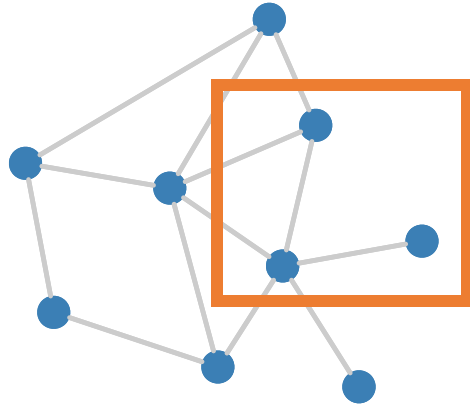
- **CNN on an image:**



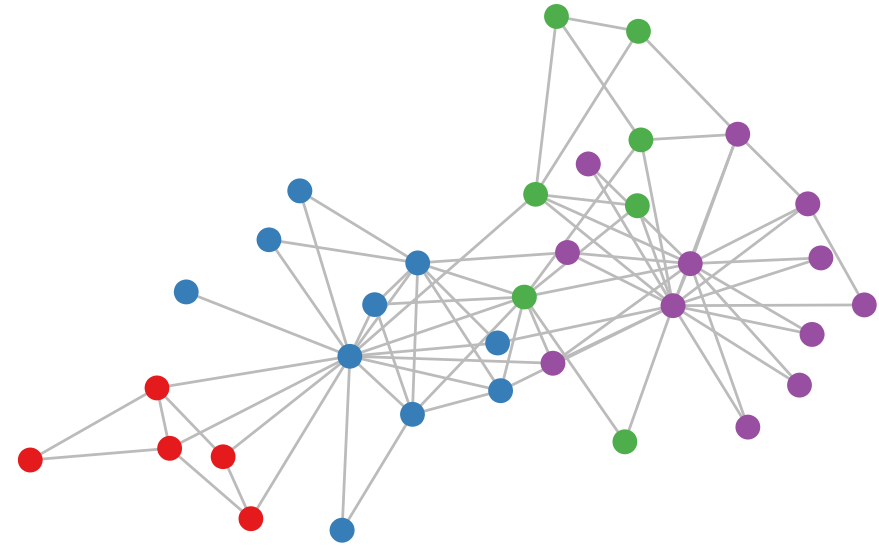
- Goal is to generalize convolutions beyond simple lattices
- Leverage node features/attributes (e.g., text, images)

Real-World Graphs

- Our graphs could look like this:



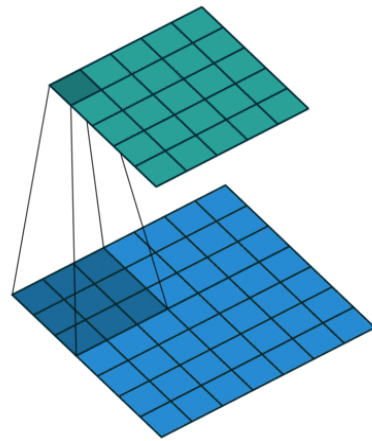
or this



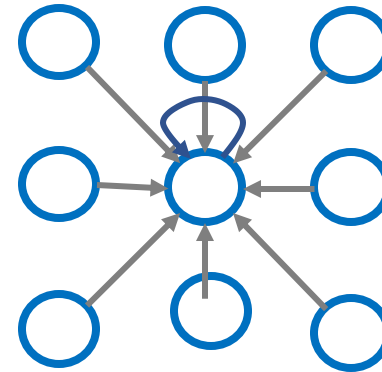
- There is no fixed notion of locality or sliding window on the graph
- Graph is permutation invariant

From Images to Graphs

Single Convolutional neural network (CNN) layer with 3x3 filter:



Image



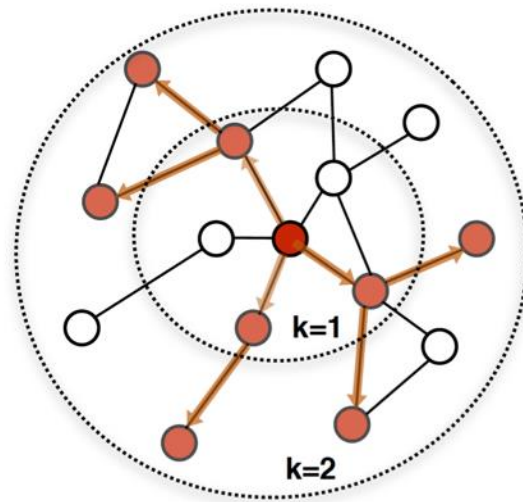
Graph

Idea: transform information at the neighbors and combine it:

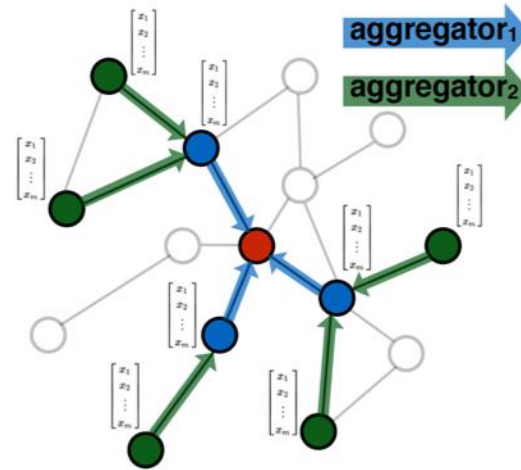
- Transform “messages” h_i from neighbors: $W_i h_i$
- Add them up: $\sum_i W_i h_i$

Graph Convolutional Networks

- **Idea:** Node's neighborhood defines a computation graph



Determine node
computation graph

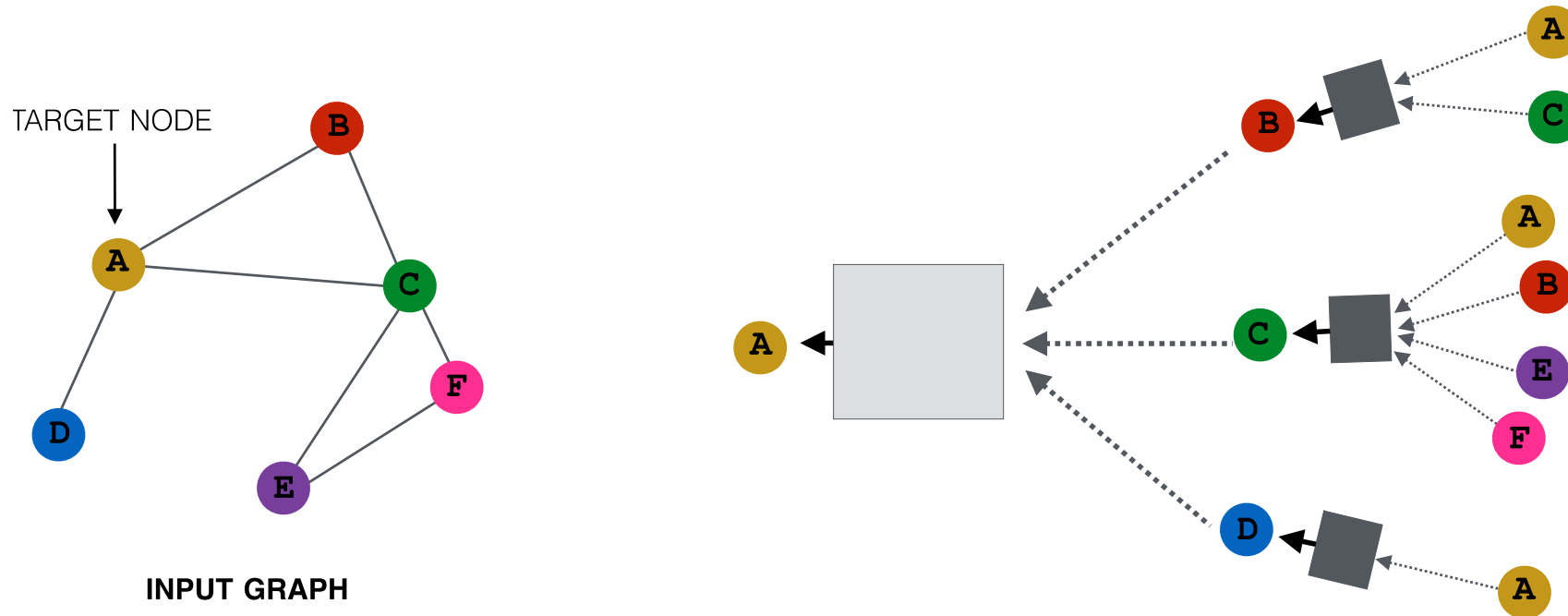


Propagate and
transform information

Learn how to propagate information across the graph to compute node features

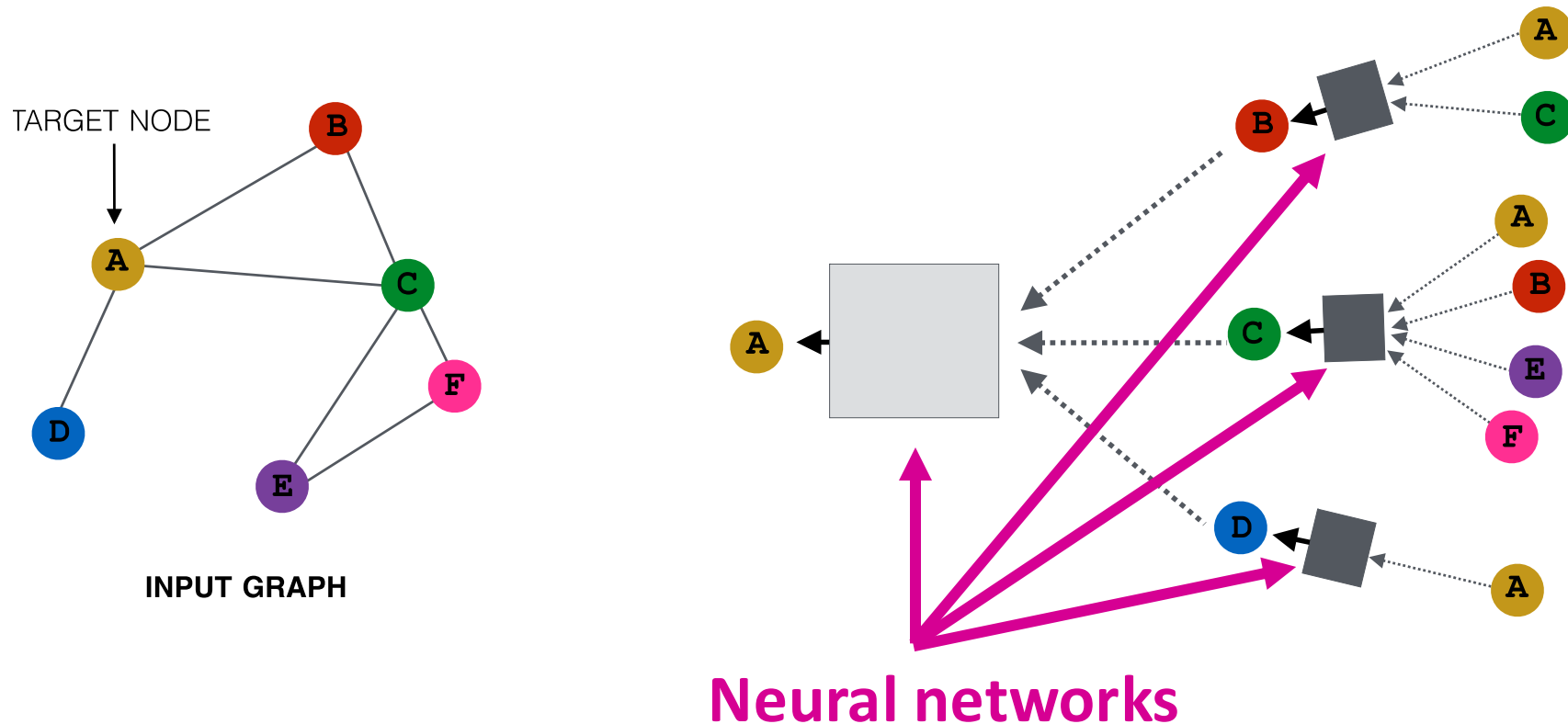
Idea: Aggregate Neighbors (1)

- **Key idea:** Generate node embeddings based on **local network neighborhoods**



Idea: Aggregate Neighbors (2)

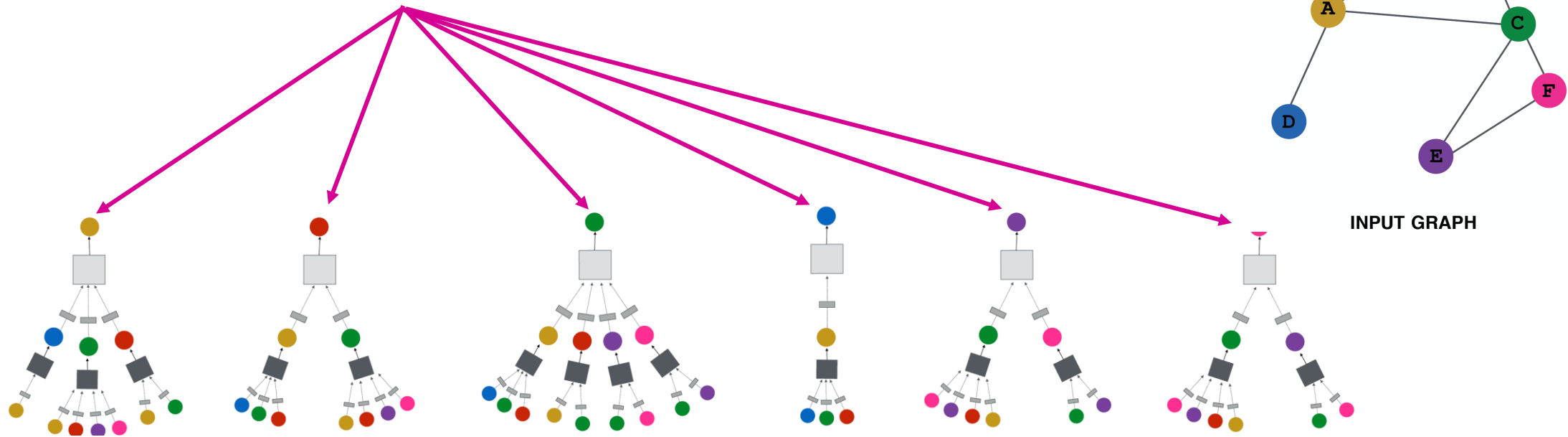
- **Intuition:** Nodes aggregate information from their neighbors using neural networks



Idea: Aggregate Neighbors (3)

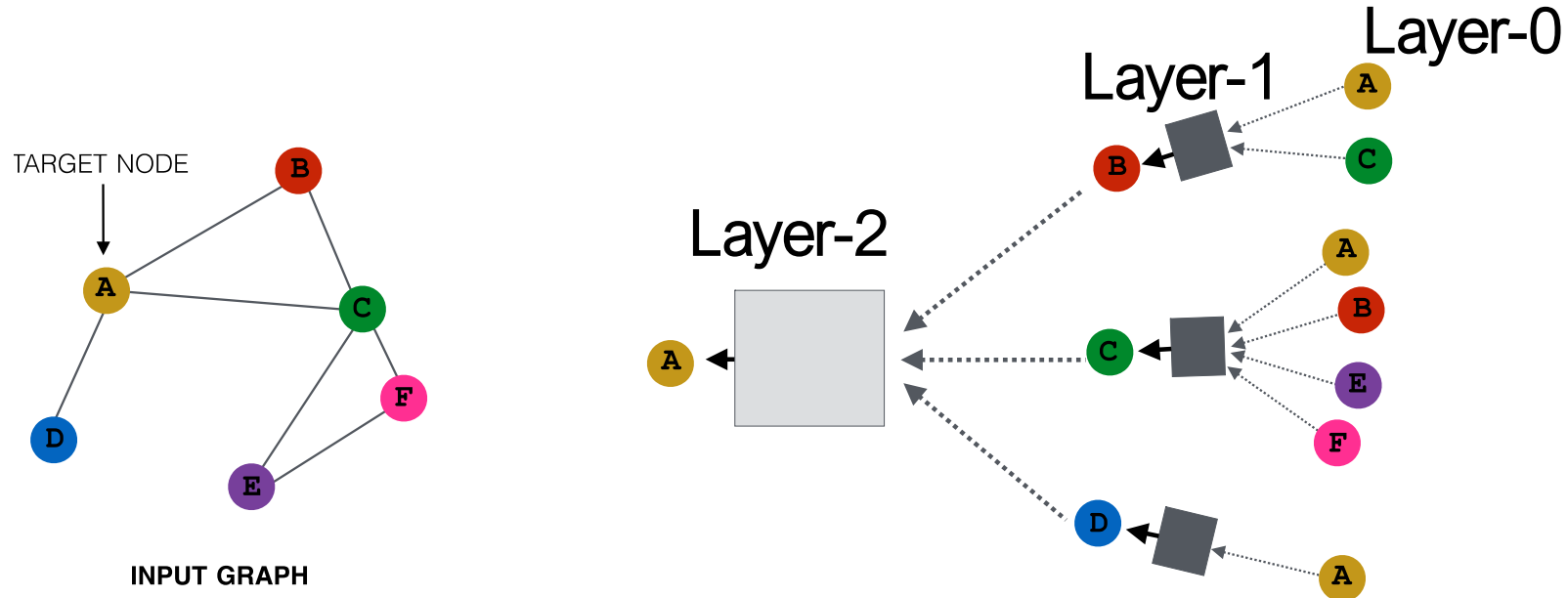
- **Intuition:** Network neighborhood defines a **computation graph**

Every node defines a computation graph based on its neighborhood!



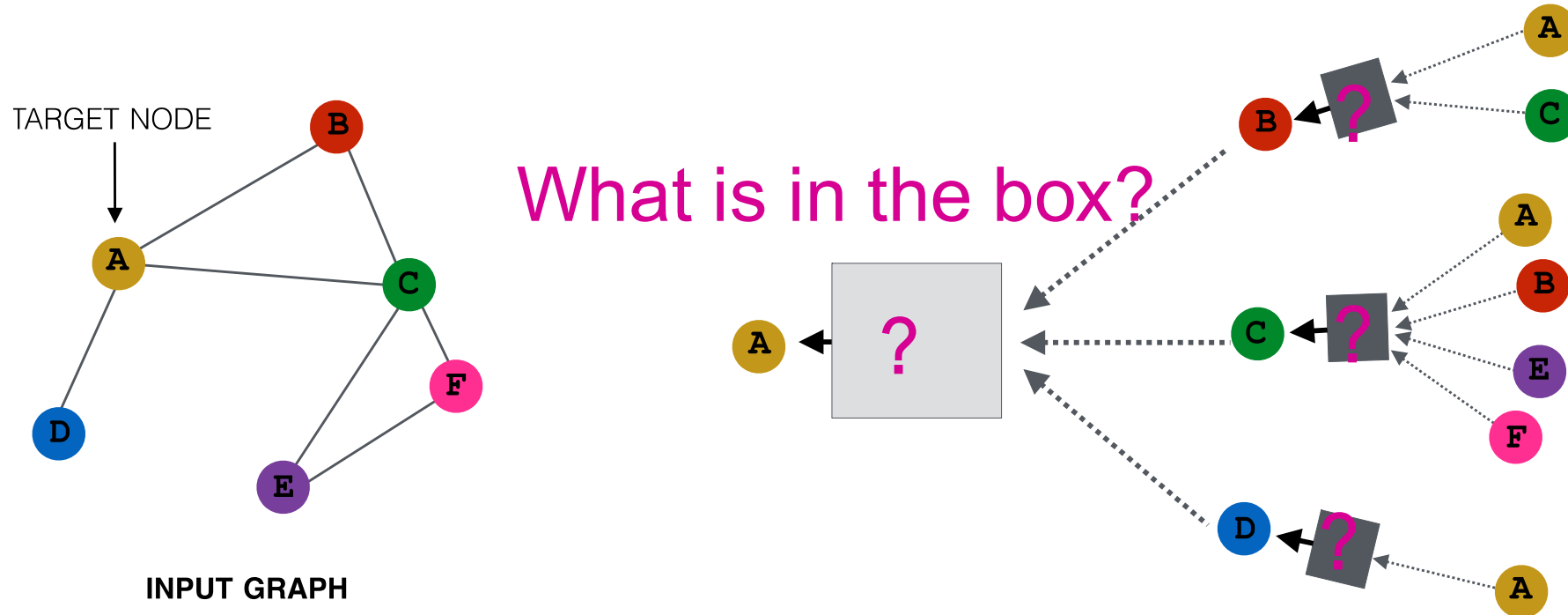
Deep Model: Many Layers

- Model can be **of arbitrary depth**:
 - Nodes have embeddings at each layer
 - Layer-0 embedding of node u is its input feature, x_u
 - Layer- k embedding gets information from nodes that are k hops away



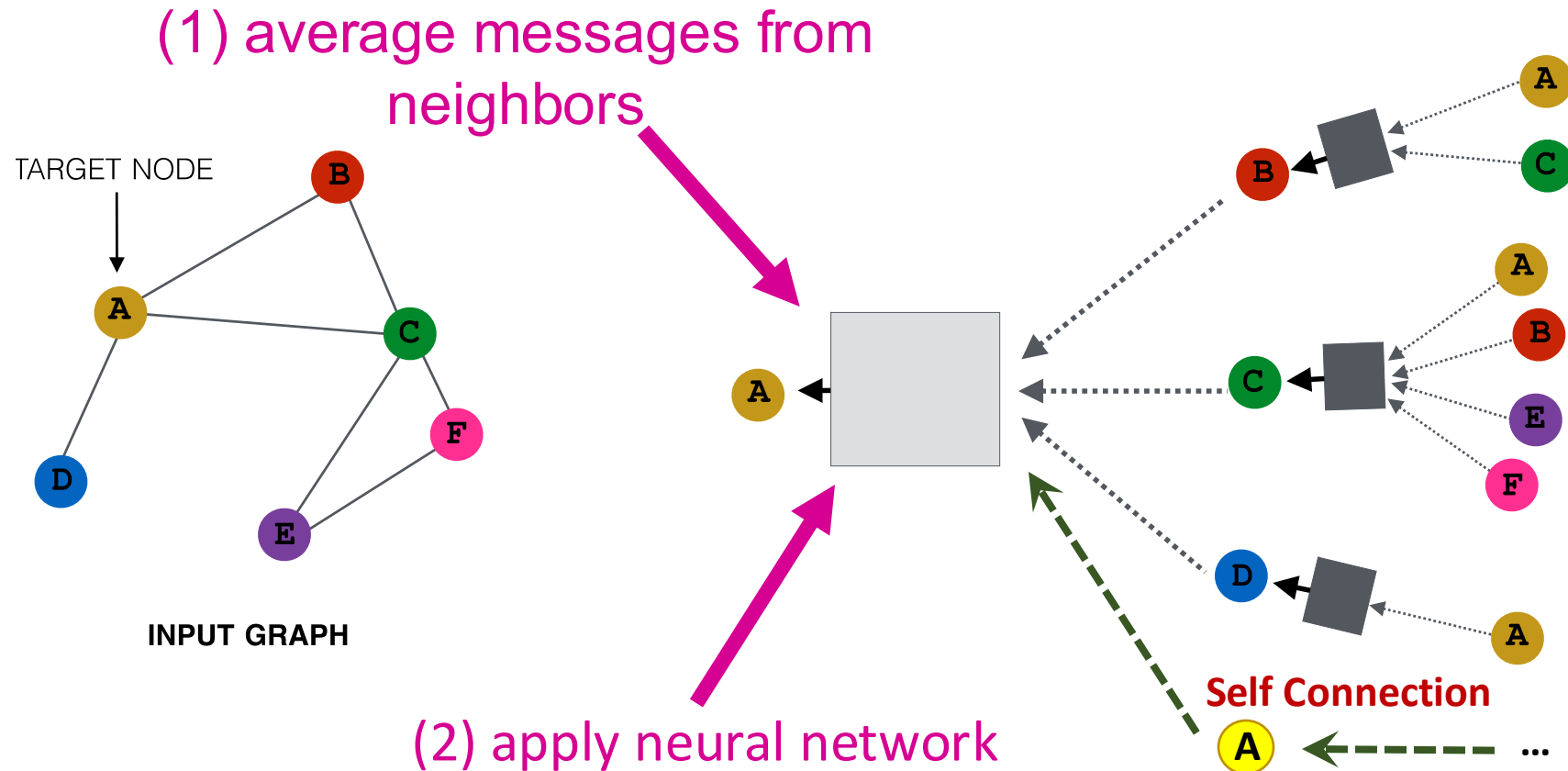
Neighborhood Aggregation (1)

- **Neighborhood aggregation:** Key distinctions are in how different approaches aggregate information across the layers



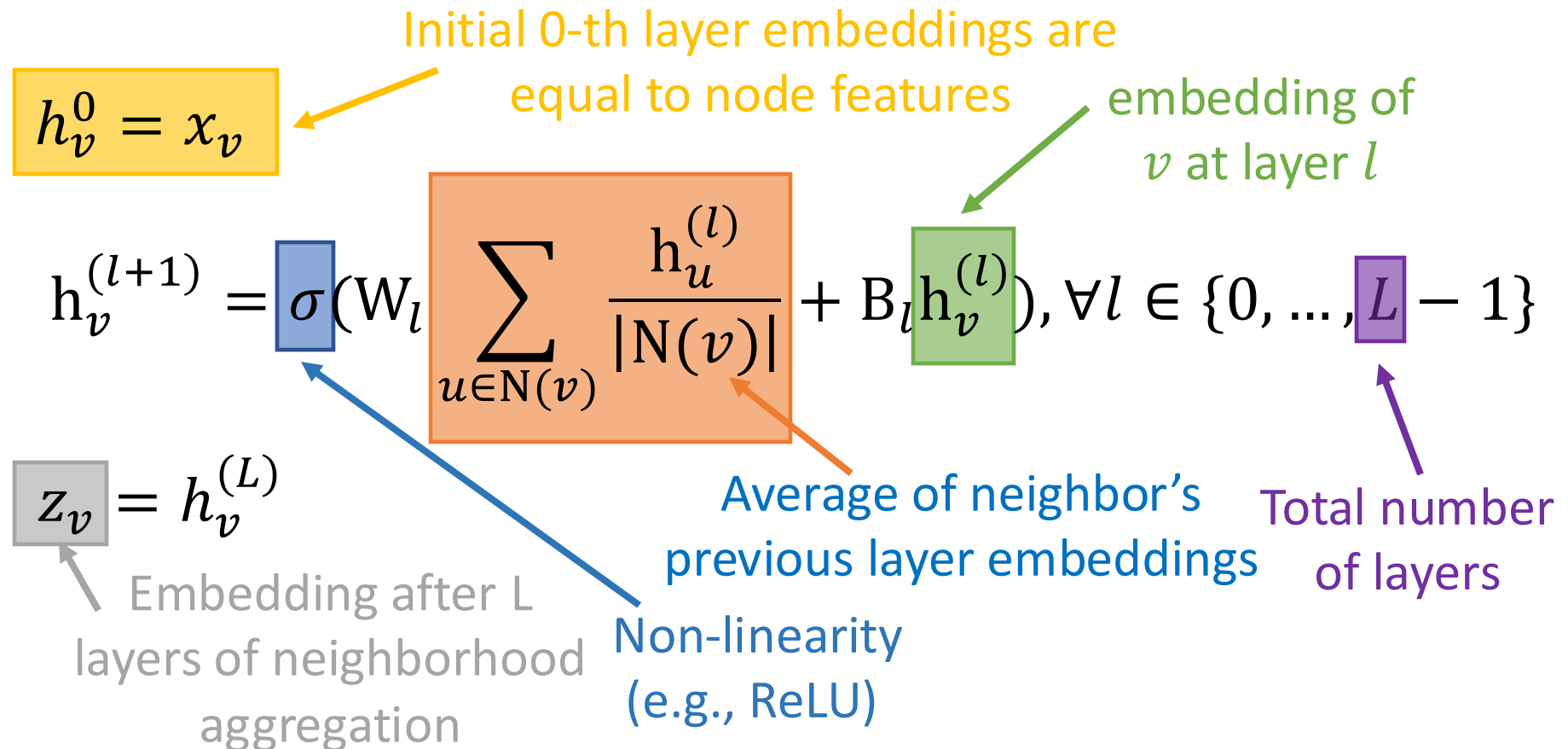
Neighborhood Aggregation (2)

- **Basic approach:** Average information from neighbors and apply a neural network



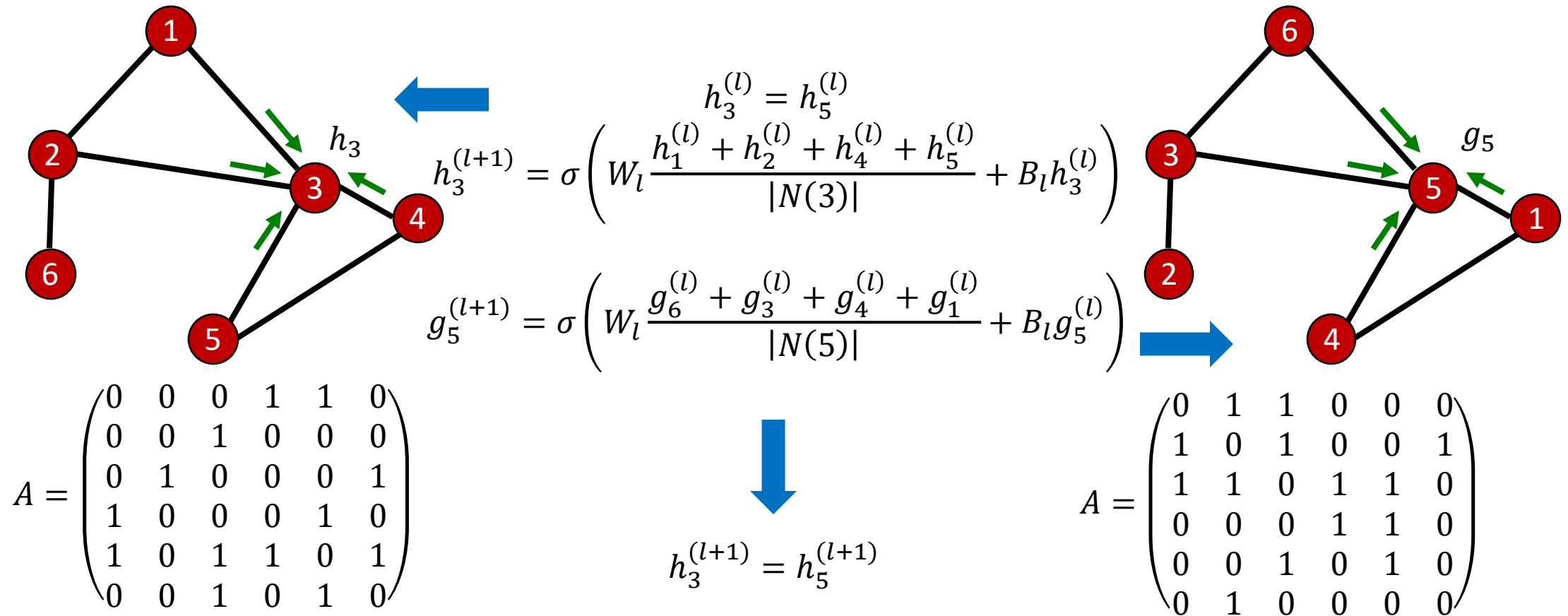
The Math: Deep Encoder

- **Basic approach:** Average neighbor messages and apply a neural network



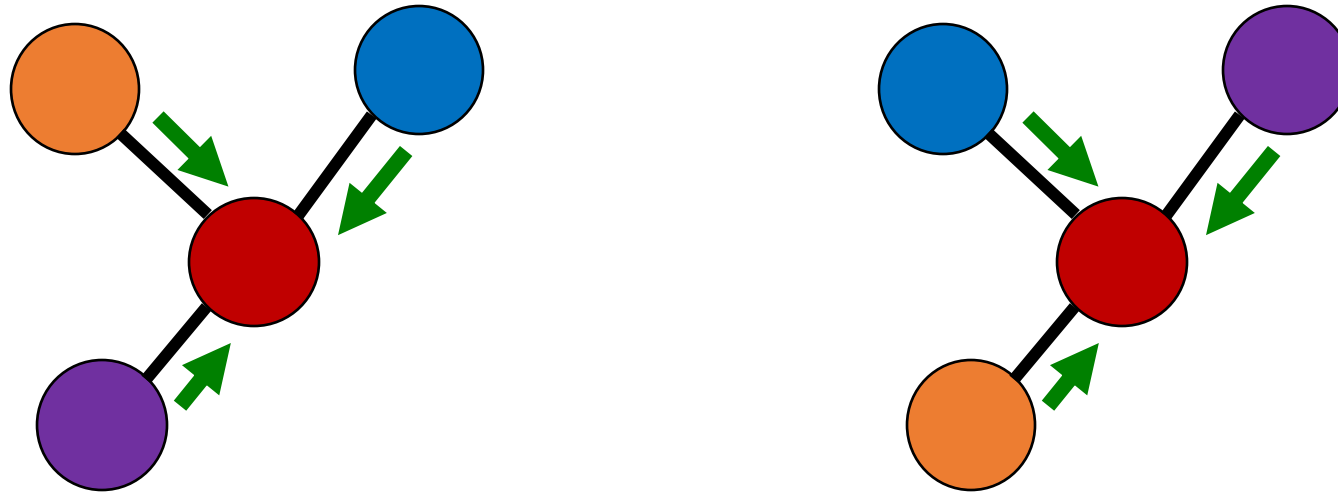
Order Invariance (Node Permutation)

- Ordering of nodes is **not** important.



Order Invariance (Neighbor Permutation)

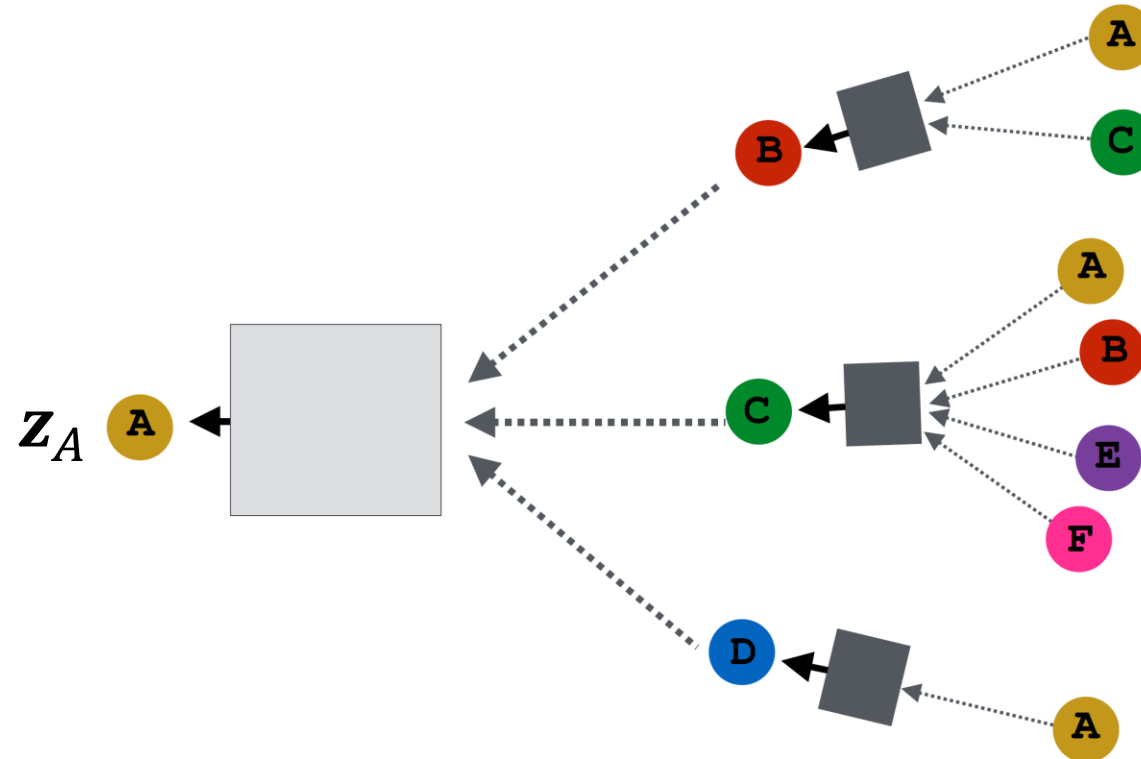
- Aggregation function **has to** be order-invariant.



- $\text{Aggr}(\bullet, \bullet, \bullet) \neq \text{Aggr}(\bullet, \bullet, \bullet)$ if the aggregation function is not order invariant. That's not what we want because the two graphs are actually the same graph.

Training the Model

How do we train the model to generate embeddings?



Need to define a loss function on the embeddings

Model Parameters

Trainable weight matrices
(i.e., what we learn)

$$\begin{aligned} h_v^{(0)} &= x_v \\ h_v^{(l+1)} &= \sigma \left(\boxed{W_l} \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + \boxed{B_l} h_v^{(l)} \right), \forall l \in \{0, \dots, L-1\} \\ z_v &= h_v^{(L)} \end{aligned}$$

Final node embedding

We can feed these **embeddings into any loss function** and run SGD to **train the weight parameters**

h_v^l : the hidden representation of node v at layer l

- W_k : weight matrix for neighborhood aggregation
- B_k : weight matrix for transforming hidden vector of self

Matrix Formulation (1)

- Many aggregations can be performed efficiently by (sparse) matrix operations

- Let $H^{(l)} = [h_1^{(l)} \dots h_{|V|}^{(l)}]^T$

- Then: $\sum_{u \in N_v} h_u^{(l)} = A_{v,:} H^{(l)}$

- Let D be diagonal matrix where $D_{v,v} = \text{Deg}(v) = |N(v)|$

- The inverse of D : D^{-1} is also diagonal:
 $D_{v,v}^{-1} = 1/|N(v)|$

- Therefore, $\sum_{u \in N(v)} \frac{h_u^{(l-1)}}{|N(v)|} \longrightarrow H^{(l+1)} = D^{-1} A H^{(l)}$

Matrix of hidden embeddings H^{k-1}



$A_{v,:}$



h_i^{k-1}

should be A^T in directed graph

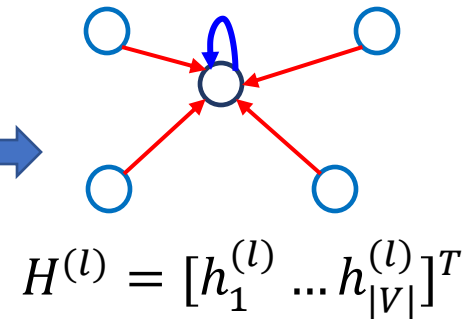
Matrix Formulation (2)

- Re-writing update function in matrix form:

$$H^{(l+1)} = \sigma(\tilde{A}H^{(l)}W_l^T + H^{(l)}B_l^T)$$

- where $\tilde{A} = D^{-1}A$

- Red: neighborhood aggregation
- Blue: self transformation (matrix)



- In practice, this implies that efficient sparse matrix multiplication can be used ($\tilde{A}$ is sparse)
- **Note:** not all GNNs can be expressed in matrix form, when aggregation function is complex

How to train a GNN

- Node embedding $\mathbf{z}_v$ is a function of input graph
- **Supervised setting**: we want to minimize the loss $\mathcal{L}$ (see also [slide](#)):

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{z}_v))$$

- $\mathbf{y}$: node label
 - $\mathcal{L}$ could be L2 if $\mathbf{y}$ is real number, or cross entropy if $\mathbf{y}$ is categorical
- **Unsupervised setting**:
 - No node label available
 - Use the graph structure as the supervision!

Unsupervised Training

- “Similar” nodes have similar embeddings

$$\mathcal{L} = \sum_{z_u, z_v} \text{CE}(y_{u,v}, \text{DEC}(z_u, z_v))$$

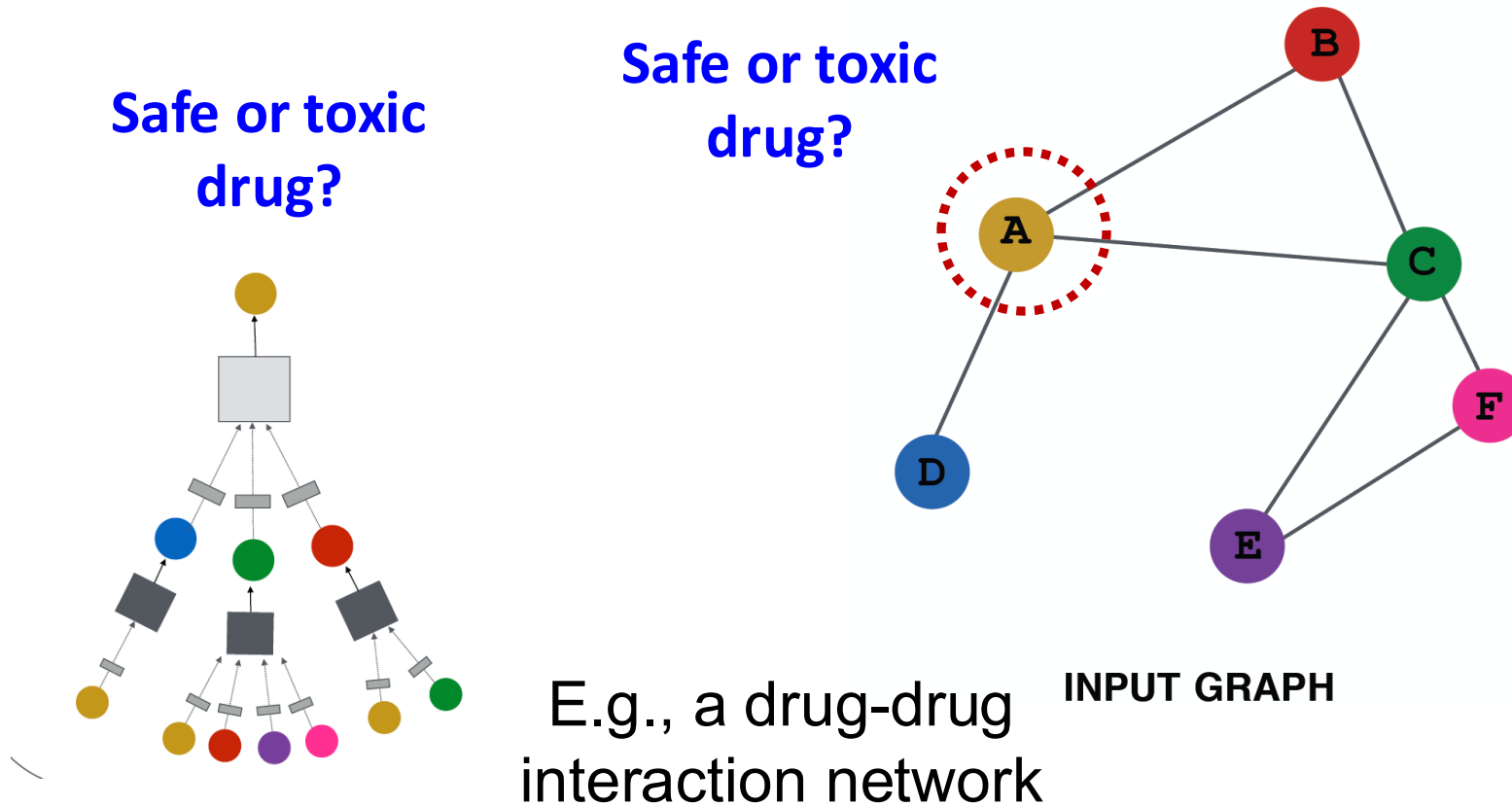
Recall in slides 15:

$$\text{CE}(\mathbf{y}, f(\mathbf{x})) = - \sum_{i=1}^C (\mathbf{y}_i \log f(\mathbf{x})_i)$$

- Where $y_{u,v} = 1$ when node u and v are **similar**
- **CE** is the cross entropy ([slide](#))
- **DEC** is the decoder such as inner product
- **Node similarity** can be anything, e.g., a loss based on (we may talk in the future):
 - **Random walks** (node2vec, DeepWalk, struc2vec)
 - **Matrix factorization**
 - **Node proximity in the graph**

Supervised Training (1)

Directly train the model for a supervised task (e.g., node classification)



Supervised Training (2)

Directly train the model for a supervised task (e.g., **node classification**)

- Use cross entropy loss ([slide](#))

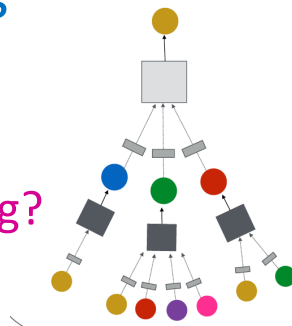
$$\mathcal{L} = \sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1 - y_v) \log(1 - \sigma(z_v^T \theta))$$

Encoder output:
node embedding

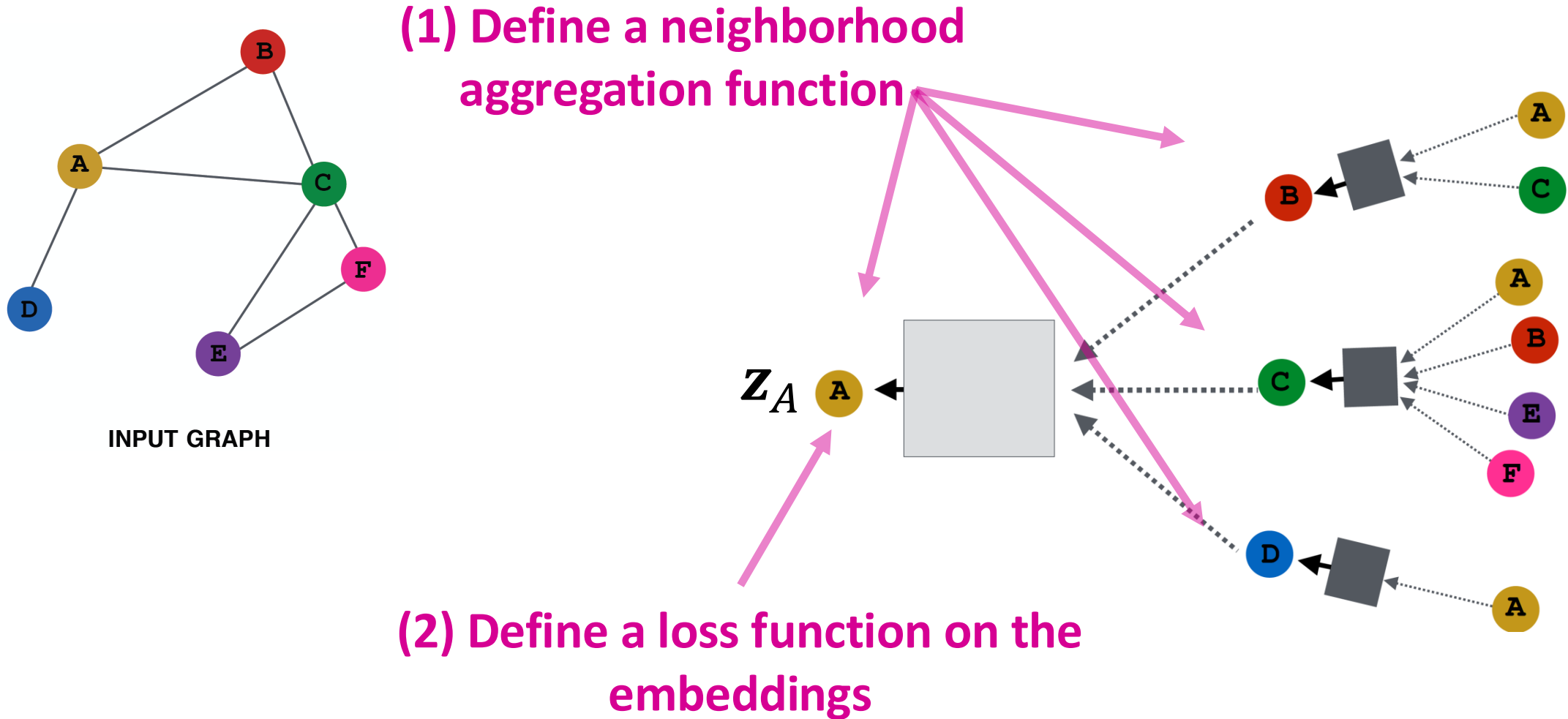
Classification
weights

Node class
label

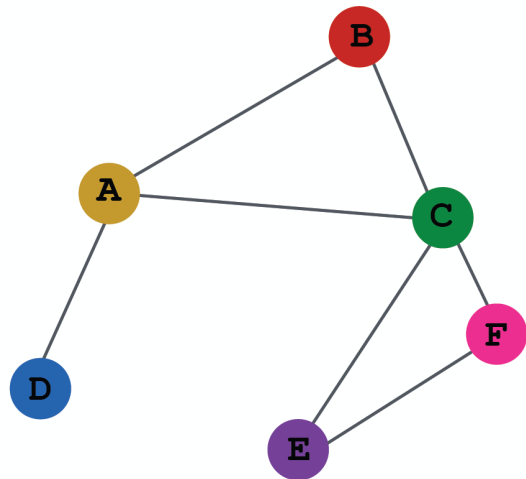
Safe or toxic drug?



Model Design: Overview (1)

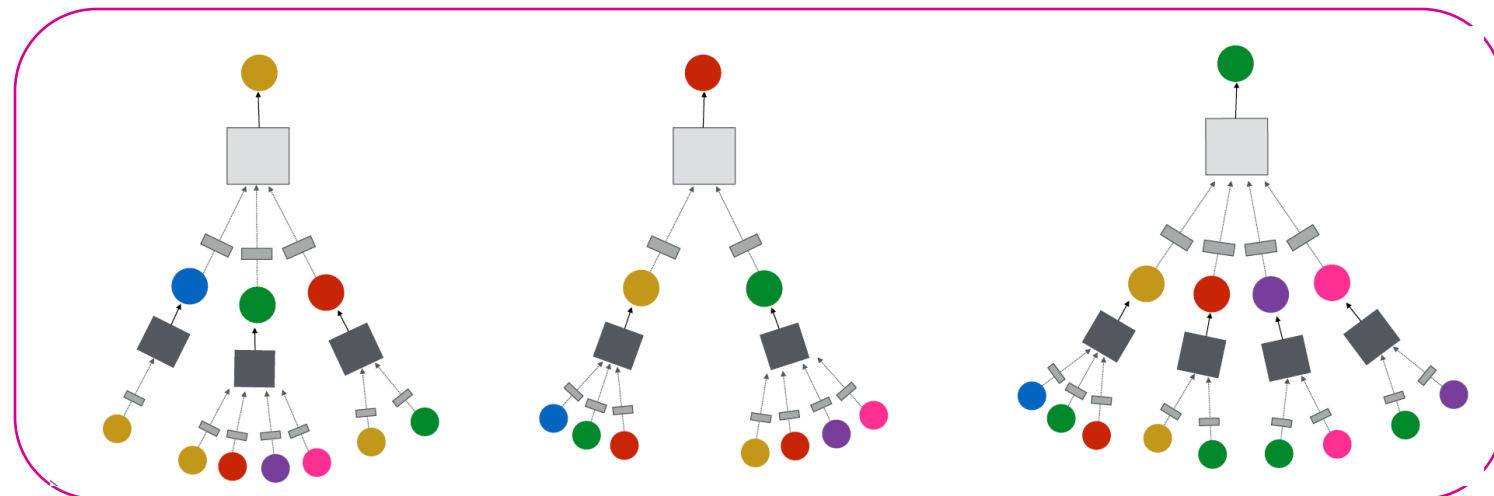


Model Design: Overview (2)

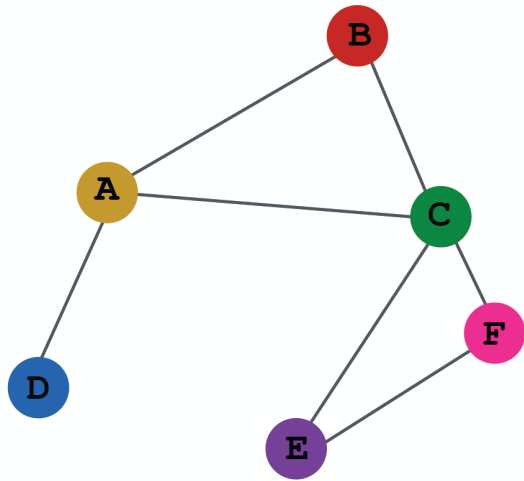


INPUT GRAPH

(3) Train on a set of nodes, i.e.,
a batch of compute graphs



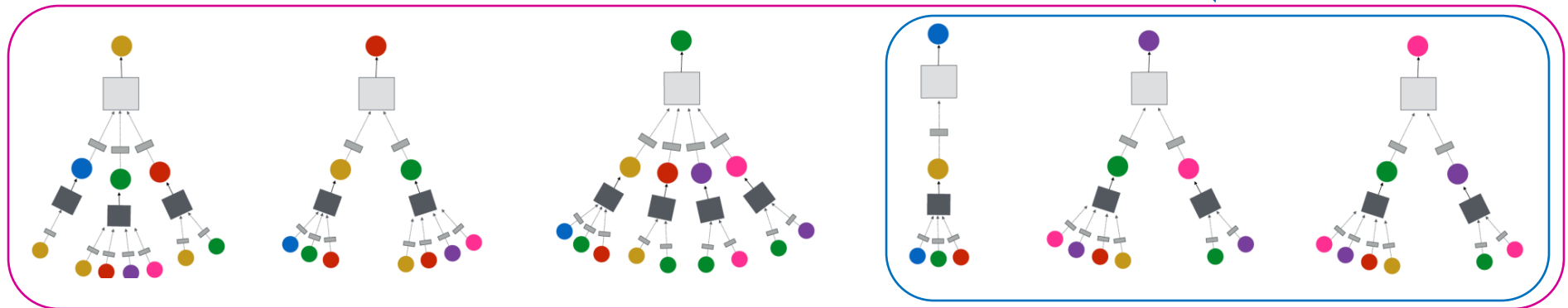
Model Design: Overview (3)



INPUT GRAPH

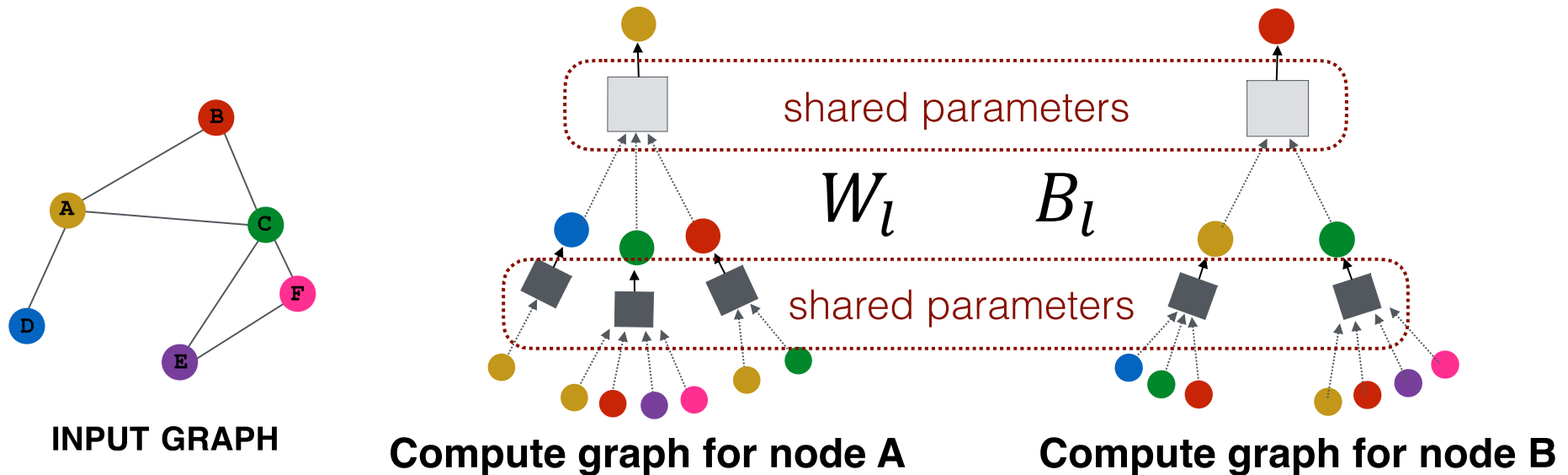
(4) Generate embeddings
for nodes as needed

Even for nodes we never
trained on!

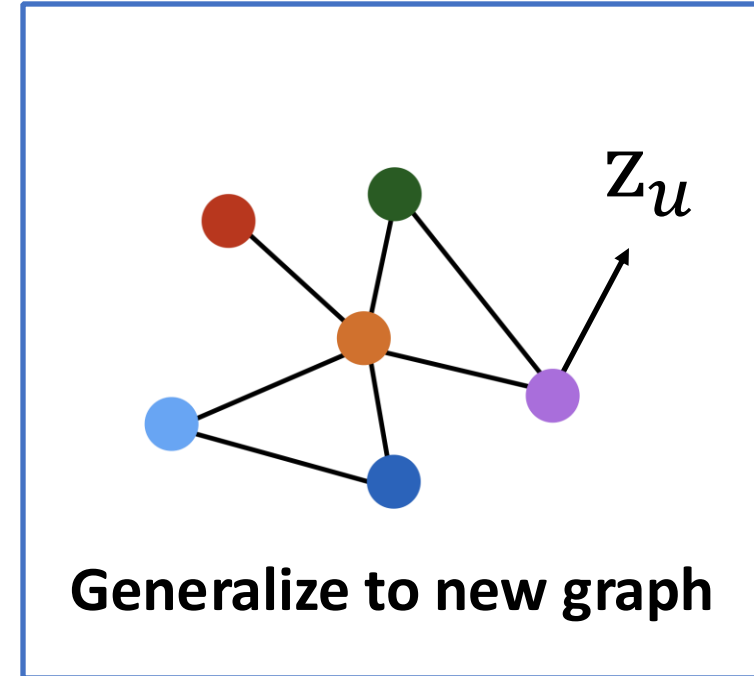
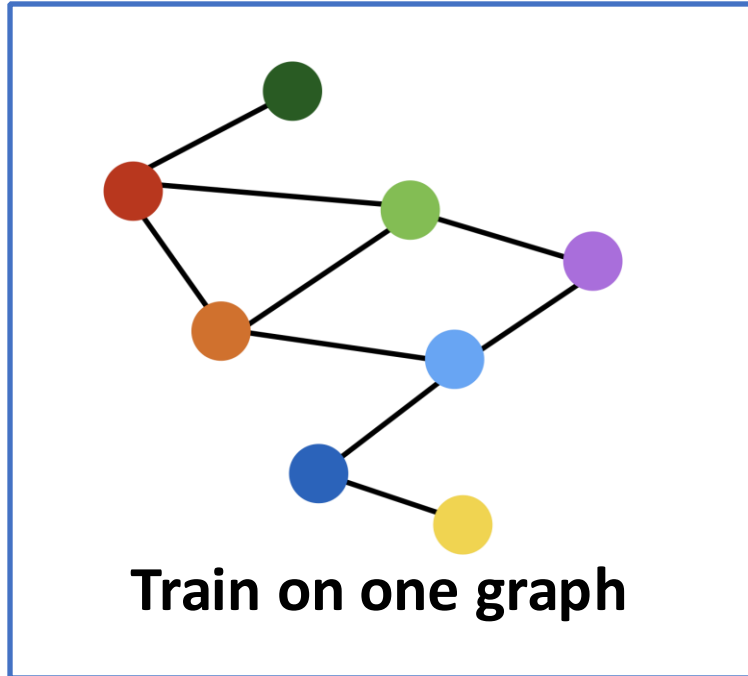


Inductive Capability

- **The same aggregation parameters are shared for all nodes:**
 - The number of model parameters is sublinear in $|V|$ and we can **generalize to unseen nodes!**



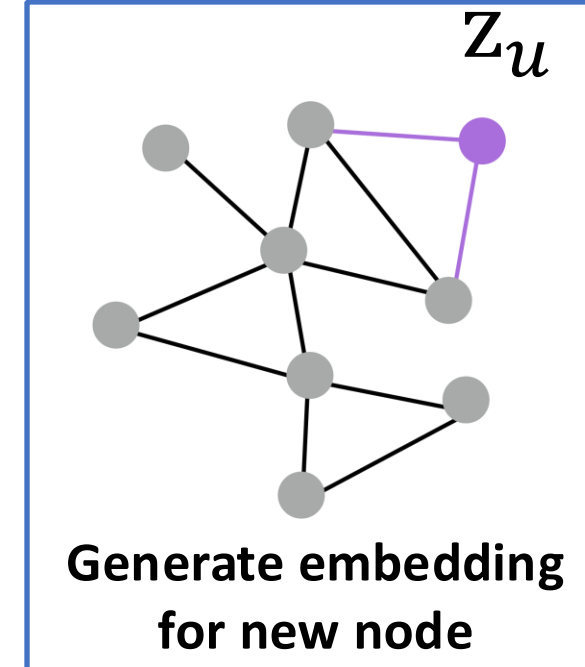
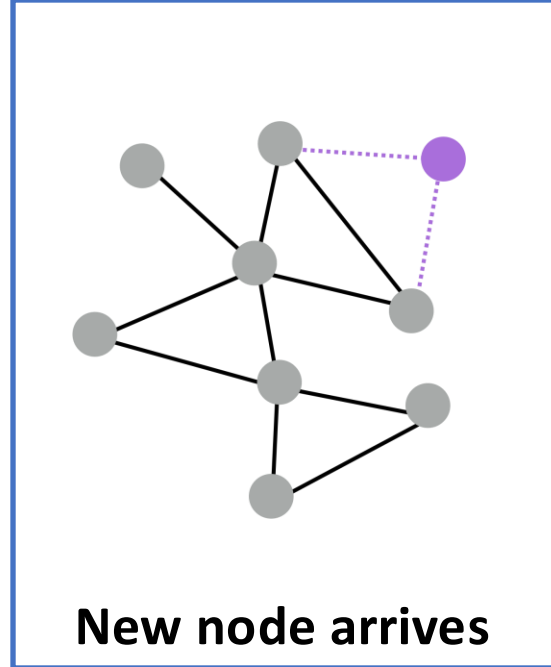
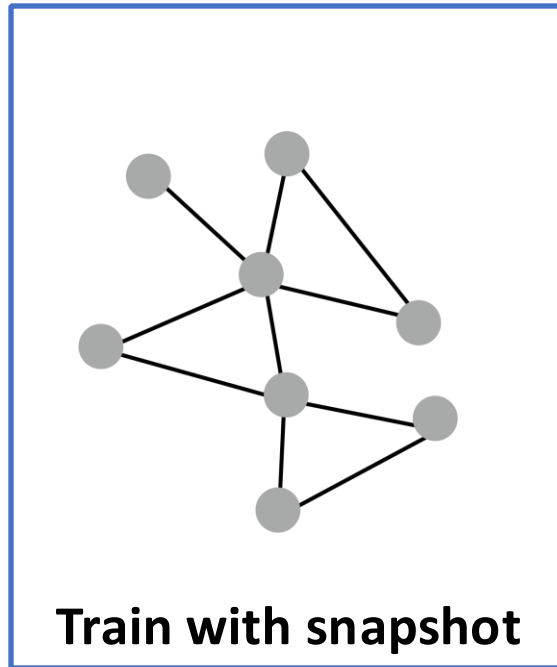
Inductive Capability: New Graphs



Inductive node embedding → Generalize to entirely unseen graphs

E.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Inductive Capability: New Nodes



- Many application settings constantly encounter previously unseen nodes:
 - E.g., Reddit, YouTube, Google Scholar
- Need to generate new embeddings “on the fly”

Summary

- **Recap: Graph Neural Networks (GNNs)** generates node embeddings by aggregating neighborhood information
 - Key distinctions between different architectures are in how they aggregate information across the layers
- **Next:** Describe GraphSAGE graph neural network architecture

Outline of Today's Lecture

1. Basics of deep learning



2. Deep learning for graphs



3. Graph Convolutional Networks and GraphSAGE

